HACKATHON MVP · ENGINEERING SPEC · v1.0

Pyramid OS

Software Documentation and Engineering Specification

A comprehensive 50+ page implementation guide for the hackathon MVP: requirements, design, architecture, data model, algorithms, APIs, and team execution plan.

Product thesis	Backstage intelligence and frontstage guest experience in one event launch-control system.
Primary demo scenario	Startup conference for 180 guests with keynote, two breakouts, coffee, equipment, QR guest registration, and launch readiness.
Recommended stack	Next.js, React, TypeScript, Supabase Postgres, Supabase Realtime, OpenAI Structured Outputs, React Flow, SVG Pyramid Twin.
Audience	Five engineers building a polished hackathon MVP and a credible future product.
Version	v1.0 — Generated 2026-06-19

Winning position: Other platforms book rooms. Pyramid OS launches experiences.

1. Document Map

This document is intentionally implementation-oriented. It gives product, design, backend, frontend, database, AI, and QA engineers the same operating model. The goal is to reduce debate during the hackathon and keep the team aligned on one ambitious but buildable product.

- Executive summary and winning strategy
- Product scope and non-goals
- Roles, permissions, and user journeys
- Detailed requirements
- Feature specifications
- Pyramid Twin design
- Guest layer design
- Equipment system
- Architecture and technical decisions
- AI architecture
- Database design
- API design
- Algorithms
- Frontend implementation details
- Realtime and notifications
- Security and permissions
- Testing and QA
- Deployment
- Hackathon execution plan
- Appendices and reference material

2. Executive summary and product thesis

Pyramid OS is an event launch-control platform for the Pyramid of Tirana. It turns an ambiguous event request into a ready-to-execute operational plan that connects spaces, availability, equipment, staff tasks, quotes, approvals, guest registration, guest navigation, and post-event learning.

The product should not be framed as venue booking software. The competitive field will likely produce calendars, forms, and inventory tables. Pyramid OS should feel like an operating system for physical experiences. The core promise is that the system answers two questions instantly: Can we make this happen? If yes, what is next?

> The MVP must show one polished end-to-end journey: a messy request from an external event organizer becomes an Event DNA fingerprint, a Pyramid Twin plan, equipment reservations, conflict auto-fixes, a quote, guest registration, and a final GO/NO-GO launch state.

Winning demo narrative

- An **external event organizer** (a client, not Pyramid staff) submits a messy startup conference request for 180 guests through the organizer request portal.
- AI Event Intake extracts structured requirements and confidence scores.
- Event DNA visualizes complexity, risk, guest journey complexity, and technical intensity.
- Pyramid Twin recommends Green Room, Blue Room, Yellow Room, and Entrance Area.
- The system reserves spaces and assets across setup, event, teardown, and return windows.
- Conflict Auto-Fixer resolves a missing wireless microphone by substituting one wired microphone.
- Power and Cable Intelligence detects missing extension cables and cable covers.
- Decision Graph explains why each room, asset, and fix was selected.
- Ops Launch Mode confirms Space, Assets, Power, Staff, Client, Guest Page, Registration, and Map as GO.
- The event is published to Guest Mode, guests register, receive QR passes, and see a guest-safe map.

Primary product pillars

Pillar	Meaning	Core features
Understand	Convert messy inputs into reliable structured event requirements.	AI intake, Event DNA, request confidence, missing-info detection
Simulate	Model how the event uses spaces, time, assets, and guest paths.	Pyramid Twin, space matcher, availability, asset movement, timeline
Protect	Detect and resolve the hidden failure points before event day.	Conflict engine, auto-fixes, power/cable intelligence, safety checks
Explain	Make every recommendation understandable and auditable.	Decision Graph, change impact, audit log, plan versions
Launch	Turn the plan into execution readiness.	Ops tasks, QR assets, launch mode, staff briefings, proposals
Experience	Publish the internal plan into a guest-friendly journey.	Guest browser, registration, QR pass, guest map, check-in dashboard

Success criteria for the hackathon

- An external event organizer (judge acting as a client) can submit a natural-language event request and see the system understand it in seconds.
- The Pyramid Twin visually lights up the recommended spaces and asset flows.
- At least one real operational conflict is detected and auto-fixed in the demo.
- Equipment is reserved with correct time windows and shown with QR tracking logic.
- The guest layer is connected to operations: publishing an event creates a public page and QR registration.

- The final screen produces an emotional memory hook: EVENT READY FOR LAUNCH.
- The system feels buildable, not fake: AI writes and explains, while deterministic code calculates availability, prices, conflicts, and reservations.

3. Scope, non-goals, and MVP boundaries

MVP scope

The MVP will be built around one excellent event scenario. It must feel complete, but it does not need to handle every possible venue policy. The correct hackathon strategy is depth in one scenario plus extensibility in the architecture.

Item	Decision / Requirement
Event types in demo	Startup conference, workshop, exhibition, performance, hackathon. Only startup conference must be fully polished.
Spaces in scope	Blue, Orange, Green, Yellow, Entrance, Corridors, Storage A, Storage B, Tech Storage, Tech Booth.
Equipment in scope	Chairs, tables, wireless microphones, wired microphones, screens, projectors, speakers, extension cables, cable covers, signage stands, registration desk.
Organizer scope	External event organizer submits a request, tracks request/ proposal status, reviews and approves the proposal, and requests changes — without access to internal operational data.
Guest scope	Browse public events, view event detail, register, receive QR ticket, open guest map, staff check-in.
AI scope	Structured intake, proposal text, task/briefing text, explanations. AI must not invent availability, prices, or inventory.

Non-goals for the hackathon

- No real payment processing.
- No full CAD-grade floor planning.
- No full 3D digital twin or AR setup overlay.
- No real RFID/NFC hardware integration.
- No production-grade supplier integrations.
- No legal safety certification. Use operational safety checks only.
- No complex multi-tenant enterprise billing system.
- No production chatbot that can act without structured data validation.

Feature sacrifice order if Guest Mode is added

The Guest layer should be added because it extends the system from backstage operations to frontstage experience. No product feature is removed from the vision, but if implementation time becomes tight, the following sacrifice order keeps the strongest demo intact.

Order	Simplify first	Keep this minimal version
1	Full Git for Events branch/rollback system	Keep simple Change Impact diff for attendee count change.
2	Power and Cable Intelligence full module	Keep one high-impact cable kit warning and fix.
3	Decision Graph dynamic builder	Keep one graph generated for the main demo.
4	Invisible Work Eraser dynamic calculation	Keep a static impact widget based on event complexity.
5	Downloadable proposal PDF	Keep a polished proposal preview if PDF export fails.

4. Users, roles, permissions, and ownership model

Pyramid OS has staff-facing, organizer-facing, and guest-facing roles. Staff roles work with sensitive operational data; the external **event organizer** initiates and tracks their own request; guests see only public event information and their own registration. Permissions must be simple for the hackathon but designed for Row Level Security and future production hardening.

> **New in this version:** the **Event Organizer** is modelled as an explicit external role. They are the outsider who *sets* the event (submits the request) and are not part of the Pyramid staff. This separates "who asks for the event" (Event Organizer / client) from "who reviews and runs it" (Event Manager / Operations staff) and from "who attends it" (Guest).

Role	Primary goals	Can access
Event Organizer *(external client — not Pyramid staff)*	Submit an event request, describe needs, track request and proposal status, review and approve the proposal, and ask for changes.	Own event requests, proposals/quotes explicitly shared with them, the status of their own event, and the public link to their published event page only.
Admin	Configure users, spaces, pricing, asset categories, and system settings.	All records inside the organization.
Event Manager	Review incoming organizer requests, approve plans, generate quotes, publish events.	Event requests, spaces, quotes, proposals, guest publication status.
Operations Manager	Coordinate setup, teardown, teams, conflicts, and launch mode.	Tasks, timelines, conflicts, Pyramid Twin operations mode.
Inventory Manager	Organize, reserve, move, scan, inspect, and return equipment.	Assets, batches, kits, reservations, movements, issues.
Technician	Handle AV, power, cables, microphones, screens, and sound checks.	Technical tasks, AV assets, power/cable warnings, issue reports.
Finance Manager	Validate quotes, margins, discounts, and proposal approval.	Quotes, quote items, pricing rules, finance approvals.
Guest	Browse public events, register, receive QR pass, navigate, check in.	Public event pages and own registration only.

Permission principles

- **Event Organizers are external (not Pyramid staff).** They can create and track only their own event requests and view proposals/quotes shared with them. They never see inventory, internal conflicts, pricing rules, other organizers' data, storage spaces, staff assignments, or operational notes.
- Operational data is private by default. Public event data is exposed only after publication.
- Guests never see inventory locations, staff names, internal conflicts, pricing rules, operational notes, or storage spaces.
- Audit logs are append-only from the application perspective.
- Staff users can belong to multiple roles; permissions are additive but must be checked server-side. The Event Organizer and Guest roles are external and are not combined with staff roles.
- Every database table that contains organization data must include org_id to support future multi-tenant separation.

Role-based mode switching in Pyramid Twin

Item	Decision / Requirement
Organizer Portal	Event Organizers do not use the Pyramid Twin. They use a simple request portal to submit a request, track its status, and review/ approve the proposal. No internal operational data is exposed.
Planning Mode	Event managers see space matches, availability, quote impacts, and recommended zones.

Operations Mode	Operations and technical teams see asset flows, tasks, cable warnings, setup status, and conflicts.
Guest Mode	Guests see only event locations, agenda, check-in, public routes, and navigation prompts.

5. End-to-end user journeys

Organizer inquiry to proposal

1. An **external Event Organizer** (a client, not Pyramid staff) submits a messy request through the organizer request portal (form or simulated inbox) and can track its status.
2. AI Event Intake extracts structured requirements and missing fields.
3. Event Manager reviews and accepts the extracted request.
4. Space Matcher recommends rooms and informal zones.
5. Inventory Reservation checks equipment and conflicts.
6. Quote Generator builds deterministic price lines.
7. AI Proposal Writer drafts client-friendly language using validated quote data.
8. The proposal is shared back to the Event Organizer; approvals are recorded and the proposal is sent or previewed.

Confirmed event to launch-ready operations

1. Event Manager confirms event and reserves spaces/assets.
2. Task Generator creates setup, AV, registration, cleaning, security, teardown, and return tasks.
3. Pyramid Twin shows recommended spaces and asset movement paths.
4. Conflict Auto-Fixer resolves or marks unresolved conflicts.
5. Ops Launch Mode evaluates readiness gates.
6. When all critical gates are GO, the event can be launched.

Guest discovery and check-in

1. Event is published from internal plan to public Guest Mode.
2. Guest browses upcoming events and opens the event detail page.
3. Guest registers and receives a QR pass.
4. Guest opens the guest-safe map and route instructions.
5. Staff scans QR at Entrance and check-in counts update operations dashboard.
6. Attendance metrics feed event replay and future planning.

Equipment reserve, move, scan, return

1. Event requirements generate required asset list and suggested kits.
2. System reserves individual assets and bulk batches across setup-event-teardown-return window.
3. Asset Movement Plan creates tasks for moving assets from storage to spaces.
4. Staff scans QR codes to mark picked, delivered, in use, returned, or issue reported.
5. Asset history creates memory for accountability and future events.

Last-minute change

1. The Event Organizer (or Event Manager on their behalf) changes attendees from 180 to 240.
2. Change Impact compares plan versions and recalculates capacity, assets, price, tasks, and conflicts.
3. Decision Graph explains why the plan changed.
4. Ops Launch Mode may drop from GO to WARNING until fixes are applied.
5. Updated guest capacity and map are published after approval.

6. Detailed product requirements

Requirements are grouped into functional, non-functional, and acceptance criteria. Every engineer should use these IDs in commits, issues, pull requests, and demo QA notes.

ID	Area	Requirement
FR-001	AI Event Intake	The system shall accept natural-language event requests and extract structured fields including event type, guests, preferred date, duration, spaces, assets, setup needs, guest needs, and missing information.
FR-002	Event request review	The system shall show extracted fields with confidence indicators and allow staff to edit before generating a plan.
FR-003	Event DNA	The system shall compute and display an event fingerprint covering people intensity, technical complexity, space complexity, asset intensity, guest journey complexity, setup risk, revenue potential, and operational risk.
FR-004	Pyramid Twin	The system shall display a clickable map of key spaces and operational locations with status overlays.
FR-005	Space matching	The system shall recommend spaces based on capacity, layout, adjacency, availability, event type, setup needs, and guest flow.
FR-006	Availability	The system shall check space conflicts across setup, event, teardown, and buffer windows, not only public event time.
FR-007	Equipment organization	The system shall organize equipment into categories, individual assets, bulk batches, and reusable kits.
FR-008	Equipment reservation	The system shall reserve assets and quantities for events with soft hold and confirmed reservation states.
FR-009	QR tracking	The system shall support QR identifiers for assets or batches and update location/status after scans or simulated scans.
FR-010	Conflict detection	The system shall detect conflicts for capacity, spaces, assets, setup overlap, teardown overlap, power/cable readiness, and guest flow.
FR-011	Conflict auto-fix	The system shall suggest and apply auto-fixes for at least missing microphones, missing chairs, cable kit shortages, and setup time conflicts.
FR-012	Decision Graph	The system shall visually explain why selected spaces, assets, and fixes were chosen.
FR-013	Change Impact	The system shall compare event plan versions and show changes to guests, spaces, assets, quote, tasks, conflicts, and feasibility.
FR-014	Quote generation	The system shall generate deterministic quote lines from pricing rules and event requirements.
FR-015	Proposal generation	The system shall generate client-facing proposal text from approved quote and plan data.
FR-016	Task planning	The system shall generate operational setup, AV, registration, cleaning, teardown, and return tasks with owners and deadlines.
FR-017	Ops Launch Mode	The system shall evaluate readiness gates and show GO/WARNING/BLOCKED states.
FR-018	Guest event browser	The system shall display published public events and let guests view details.
FR-019	Guest registration	The system shall allow guests to register for public events and receive a QR pass.
FR-020	Guest map	The system shall provide a guest-safe map and route for event spaces and agenda locations.
FR-021	Check-in dashboard	The system shall update check-in counts after QR scan or simulated scan.

FR-022	Audit trail	The system shall record important decisions, approvals, conflicts, fixes, reservations, and publication actions.
FR-023	Realtime dashboards	The system shall update Twin, task, inventory, and check-in views when relevant database records change.
FR-024	Invisible Work Eraser	The system shall estimate manual coordination steps and time saved based on the generated plan.
FR-025	Organizer request portal	The system shall let external **Event Organizers** submit event requests, track request and proposal status, review and approve a shared proposal, and submit change requests — without access to internal operational data (inventory, conflicts, pricing rules, storage, or other organizers' requests). This supports the existing AI intake flow; it does not replace staff review (FR-002).

Non-functional requirements

ID	Name	Requirement
NFR-001	Demo responsiveness	Core demo actions should respond within 2 seconds when using seeded data, except AI calls which may stream or show progress.
NFR-002	Determinism	Availability, pricing, inventory counts, reservations, launch gates, and conflicts must be calculated by code, not invented by AI.
NFR-003	Usability	Non-technical staff and external organizers must understand the main plan from Twin, status labels, and plain-language explanations.
NFR-004	Security	Guests and external organizers must not access private staff or inventory information. Server-side authorization is mandatory.
NFR-005	Auditability	Important state transitions must be reconstructable from logs and plan versions.
NFR-006	Resilience	The live demo must have fallback seeded outputs if external AI calls fail.
NFR-007	Extensibility	New spaces, assets, event types, and pricing rules should be added by data, not hard-coded UI branches.
NFR-008	Accessibility	Organizer request, guest registration, and event pages should be keyboard-accessible, high-contrast, and clear on mobile.

7. Feature specifications and acceptance criteria

AI Event Intake

Goal: Convert messy requests into structured requirements.

Implementation requirements

- Input can be pasted natural language or a simulated email/WhatsApp message, and may be submitted by an external Event Organizer through the request portal.
- Output is validated JSON with event type, guest count, date preference, duration, space needs, asset needs, budget, notes, missing fields, and confidence.
- Staff can edit extracted fields before planning.
- If AI fails, demo falls back to a pre-seeded parsed request.

Acceptance criteria

- Structured JSON appears after request submission.
- Missing or ambiguous fields are displayed.
- No quote or reservation is created before staff review.

Event DNA Fingerprint

Goal: Visualize event complexity before planning.

Implementation requirements

- Compute people intensity from guests and expected arrival pressure.
- Compute technical complexity from AV, screen, livestream, microphones, power, and cables.
- Compute space complexity from number of zones and adjacency needs.
- Compute guest journey complexity from public registration, agenda changes, and room movement.
- Show a radar/chart/fingerprint visualization plus plain-language explanation.

Acceptance criteria

- DNA appears within the Understand stage.
- Changing attendees or equipment updates the DNA.
- At least five dimensions are visible.

Pyramid Twin Command Center

Goal: Show a live map of spaces, statuses, assets, tasks, and guests.

Implementation requirements

- Represent Blue, Orange, Green, Yellow, Entrance, Corridors, Storage, and Tech Storage as clickable nodes/zones.
- Planning Mode shows recommendations and availability.
- Operations Mode shows assets, tasks, conflicts, and power/cable warnings.
- Guest Mode hides internal data and shows navigation.
- Use SVG for MVP and optionally Spline/Higgsfield for presentation visuals only.

Acceptance criteria

- Clicking a zone opens detail panel.
- Status colors change based on reservations/conflicts.
- Guest Mode hides internal conflicts and inventory.

Space Matcher

Goal: Recommend spaces and informal zones.

Implementation requirements

- Score each space on capacity fit, availability, layout fit, adjacency, guest flow, setup feasibility, and equipment distance.
- Support multi-space plans for keynote, breakouts, coffee, registration, and spillover.
- Return reasons, not only scores.

Acceptance criteria

- Green Room is recommended for the startup keynote.
- Entrance is recommended for coffee/registration.
- Blue/Yellow are recommended for breakouts.

Equipment OS

Goal: Organize, reserve, move, scan, inspect, and return operational assets.

Implementation requirements

- Support asset categories, individual assets, bulk batches, kits, reservations, movements, issues, and scans.
- Reserve equipment across setup-event-teardown-return window.
- Allow substitution and external rental suggestions when shortages occur.
- Show asset journey timeline and QR scan detail page.

Acceptance criteria

- Missing wireless microphone conflict is detected.
- Auto-fix reserves wired microphone substitute.
- QR asset page shows current location, destination, event, and actions.

Conflict Auto-Fixer

Goal: Turn detected issues into suggested actions.

Implementation requirements

- For asset shortage, suggest substitute, external rental, reduced setup, or time change.
- For capacity conflict, suggest larger space, overflow, reduced layout, or split agenda.
- For power/cable conflict, suggest cable kit and cable covers.
- Applying a fix must update reservations, tasks, and launch gate states.

Acceptance criteria

- At least one auto-fix button works end to end.
- After fix, conflict status changes to resolved.
- Audit log records the fix.

Power and Cable Intelligence

Goal: Catch hidden technical risks ignored by basic booking tools.

Implementation requirements

- Calculate powered-device count and nearby power-point capacity.
- Estimate cable kit needs based on stage, screen, speakers, livestream table, and guest paths.
- Generate cable safety tasks and asset reservations.

Acceptance criteria

- Demo shows power readiness warning.
- Adding Cable Kit A changes Power gate from WARNING to GO.

Decision Graph

Goal: Explain the plan visually.

Implementation requirements

- Graph nodes represent request facts, constraints, decisions, conflicts, fixes, and tasks.
- Edges explain why a decision follows from a fact or constraint.
- Graph can be generated from planning outputs; AI may summarize but not invent nodes.

Acceptance criteria

- Click "Why this plan" to see node graph.
- Graph includes guest count, Green Room decision, breakout rooms, microphone conflict, and fix.

Change Impact / Plan Versions

Goal: Make last-minute changes safe.

Implementation requirements

- Save plan snapshots as JSON versions.
- Diff old and new plan for guests, spaces, assets, quote, conflicts, tasks, and feasibility.
- For MVP, implement before/after diff and keep advanced rollback as stretch.

Acceptance criteria

- Changing guests 180 to 240 updates feasibility, assets, quote, and conflicts.
- UI shows Plan v1 to Plan v2 diff.

Proposal and Operations Pack

Goal: Generate client and internal execution outputs.

Implementation requirements

- Client proposal contains summary, spaces, schedule, included equipment, price lines, assumptions, and approval section, and is shared back to the Event Organizer for approval.
- Operations pack contains setup tasks, AV checklist, asset movement plan, power/cable checklist, guest route, and teardown plan.
- AI writes prose only after validated data is passed in.

Acceptance criteria

- Proposal preview is generated.
- Ops pack tasks are generated and linked to spaces/assets.

Ops Launch Mode

Goal: Final readiness screen.

Implementation requirements

- Evaluate Space, Assets, Power, Staff, Proposal, Client, Guest Page, Registration, Map, Safety, and Conflicts gates.
- Gate states are GO, WARNING, or BLOCKED.
- Critical BLOCKED gates prevent EVENT READY FOR LAUNCH.

Acceptance criteria

- After fixes, final screen says EVENT READY FOR LAUNCH.
- If microphone conflict is reopened, Assets gate changes to WARNING/BLOCKED.

Guest Journey Mode

Goal: Publish internal plan into a public event experience.

Implementation requirements

- Published events appear in a guest browser.
- Guests can view details, register, receive QR pass, and open guest-safe map.
- Staff check-in dashboard updates after QR scan or simulated scan.
- Guest registration can feed attendance confidence and crowd pressure calculations.

Acceptance criteria

- Published event appears publicly.
- Guest registration creates a ticket.
- Check-in changes checked-in count.

Invisible Work Eraser

Goal: Show measurable operational impact.

Implementation requirements

- Estimate emails, calls, spreadsheet checks, inventory checks, meetings, and hours avoided.
- Calculation can use event complexity assumptions for MVP.
- Display as an impact widget on Launch page.

Acceptance criteria

- Widget appears after plan generation.
- Values scale with event complexity.

8. Pyramid Twin design

The Pyramid Twin is a simplified operational model of the building. It should not attempt to be a perfect architectural model. It is a task-focused map that answers what is happening where, what needs to move, and what is blocking launch readiness.

Twin layers

Layer	Visible to	Purpose	Example
Spaces	Staff and guests	Show rooms and public zones.	Green, Blue, Yellow, Entrance.
Availability	Staff	Show open, reserved, blocked, setup, teardown.	Orange booked 09:00–13:00.
Asset flows	Staff	Show equipment moving from storage to event spaces.	Storage A to Green Room: 150 chairs.
Conflicts	Staff	Show issue pins and suggested fixes.	Mic shortage; cable kit missing.
Power/cables	Ops and tech	Show cable routes and power readiness.	Cable covers required across guest path.
Tasks	Staff	Show pending setup and teardown work.	Sound check due 09:30.
Guest route	Guests	Show public route and agenda locations.	Entrance to Registration to Green Room.
Check-in heat	Staff	Show live or predicted arrival pressure.	Entrance pressure high at 09:45.

Recommended SVG map structure

```
<PyramidTwin mode="operations" eventId="event_startup_001">
  <Zone id="entrance" label="Entrance" status="reserved" />
  <Zone id="green" label="Green Room" status="recommended" />
  <Zone id="blue" label="Blue Room" status="reserved" />
  <Zone id="yellow" label="Yellow Room" status="reserved" />
  <Zone id="orange" label="Orange Room" status="available" />
  <Zone id="tech_storage" label="Tech Storage" status="asset-source" />
  <AssetFlow from="tech_storage" to="green" label="3 wireless mics + screen" />
  <ConflictPin zone="green" severity="medium" label="1 mic substituted" />
</PyramidTwin>
```

Twin state model

```
type TwinMode = 'planning' | 'operations' | 'guest';
type ZoneStatus = 'available' | 'recommended' | 'reserved' | 'setup' | 'blocked' | 'conflict';
type TwinZone = {
  id: string;
  name: string;
  floor: 0 | -1;
  kind: 'room' | 'entrance' | 'corridor' | 'storage' | 'technical';
  capacity: number;
  status: ZoneStatus;
  publicVisible: boolean;
  x: number; y: number; width: number; height: number;
};
type AssetFlow = {
  id: string;
  fromLocationId: string;
  toSpaceId: string;
  assetLabel: string;
  quantity?: number;
  status: 'planned' | 'picked' | 'in_transit' | 'delivered' | 'returned';
};
```

9. Guest layer and frontstage experience design

Guest Mode is the public face of Pyramid OS. It must remain tightly connected to operations. A published event is not manually recreated as a separate website; it is derived from the approved operational plan. This is the key product advantage.

Guest feature requirements

Feature	Guest benefit	Operational benefit	MVP implementation
Public event browser	Guests discover upcoming public events.	Event managers publish from the same approved plan.	Next.js public route /events.
Event detail page	Guests see date, agenda, spaces, capacity, and description.	Public content reflects approved rooms and timing.	Dynamic route /events/[slug].
Registration	Guests reserve a spot.	Registration counts inform capacity and check-in readiness.	Simple form stored in guest_registrations.
QR ticket	Guests receive check-in pass.	Staff can scan or simulate check-in.	QR code contains ticket token.
Guest map	Guests know where to go.	Reduces wayfinding questions and queue pressure.	Twin Guest Mode with route steps.
Check-in dashboard	Faster entry.	Live registered/checked-in/remaining capacity view.	Realtime or refresh-based counter.

Guest privacy boundaries

- Guest routes cannot reveal storage rooms, staff-only corridors, internal conflict notes, asset locations, pricing details, staff assignments, or internal approvals.
- Public event pages show only fields explicitly copied into event_publications or derived from published agenda items.
- Guest registration records are visible to staff but each guest can access only their own ticket via a signed or random token.
- Check-in updates should not expose full guest lists publicly.

Guest journey data model interaction

```
Event -> EventPublication -> AgendaItems -> GuestRegistrations -> GuestTickets -> GuestCheckins
                                         -> GuestRouteSteps -> CheckInDashboard -> EventReplay ->
PyramidMemory
```

10. Equipment organization, reservation, and tracking

Equipment is the operational backbone of the system. It must be organized in a way that matches real venue work: some assets are serialized and tracked individually; others are tracked in bulk batches; many events use reusable kits.

Four-level equipment organization

Level	Model	Examples	Why it exists
1. Category	asset_categories	Chairs, tables, wireless microphones, screens, cable covers.	Defines common properties and replacement categories.
2. Individual asset	assets	Wireless Mic 03, Projector 01, Screen 01.	Needed for valuable or technical equipment.
3. Bulk batch	asset_batches	Chair Stack A: 80 chairs, Table Stack B: 15 tables.	Avoids scanning 150 chairs one by one.
4. Kit	asset_kits and asset_kit_items	Conference Kit, Cable Safety Kit, Registration Kit.	Makes repeated event setups fast and reusable.

Equipment statuses

Status	Meaning	Allowed transitions
Available	Can be reserved.	soft_hold, reserved, maintenance, retired
Soft hold	Temporarily held while proposal is pending.	reserved, released, cancelled
Reserved	Confirmed for an event.	picked, cancelled, released
Picked	Collected from storage.	in_transit, returned, missing
In transit	Moving from one location to another.	in_use, returned, missing
In use	Currently assigned to active event.	returned, missing, issue_reported
Returned	Back from event.	needs_inspection, available
Needs inspection	Must be checked before reuse.	available, maintenance
Maintenance	Not available due to repair.	available, retired
Missing	Expected but not found.	available, maintenance, retired
Retired	No longer active.	none

Reservation logic

```
1. Build requirement list from event request and selected kits.
2. Expand kits into category-level requirements.
3. Calculate reservation window = setup_start to teardown_end + return_buffer.
4. For serialized assets, choose available assets with no overlapping reservations.
5. For bulk batches, reserve quantities from batches by closest location and sufficient condition.
6. If shortage exists, create conflict and search replacement_category_id.
7. If replacement exists, propose auto-fix and update task list.
8. After approval, convert soft holds to confirmed reservations.
9. Generate asset movement tasks and Pyramid Twin asset flows.
10. Update launch gates based on reservation, scan, movement, and issue states.
```

Demo equipment scenario

Item	Decision / Requirement
Requirement	4 wireless microphones, 150 chairs, 15 tables, 1 screen, 1 projector, 2 speakers, cable kit.

Seeded conflict	Wireless Mic 04 is already reserved for Robotics Workshop.
Auto-fix	Reserve Wireless Mic 01-03 and Wired Mic 01. Mark conflict resolved.
QR demo	Scan Wireless Mic 03 and mark it delivered to Green Room.
Twin visual	Show Tech Storage to Green Room and Storage A to Green Room asset flows.

11. System architecture and technical decisions

The architecture is intentionally pragmatic. A single Next.js application with Supabase Postgres and a small AI service layer is enough for the hackathon, while preserving a clean separation between UI, application services, deterministic planning logic, and AI-assisted generation.

Architecture diagram

```
Browser / Mobile
  |-- Organizer UI: Submit request, track status, review proposal
  |-- Staff UI: Understand, Simulate, Protect, Explain, Launch
  |-- Guest UI: Browse, Register, Ticket, Map
  |
  v
Next.js App Router
  |-- Server Actions for form mutations
  |-- Route Handlers for API endpoints and QR/ticket access
  |-- React components for Twin, Graph, Timeline, Launch Mode
  |
  +--> Planning Engine (deterministic TypeScript services)
  |     |-- space matcher
  |     |-- availability checker
  |     |-- equipment reservation
  |     |-- conflict detector
  |     |-- auto-fix engine
  |     |-- feasibility and launch gates
  |
  +--> AI Orchestrator
  |     |-- structured event intake
  |     |-- proposal writing
  |     |-- explanation summaries
  |     |-- task/brief generation
  |
  +--> Document/QR Generator
  |
  v
Supabase
  |-- Postgres tables
  |-- Row Level Security policies
  |-- Realtime subscriptions
  |-- Storage for proposals and generated packs
```

Technology decisions

Decision	Choice	Rationale	Fallback
Frontend framework	Next.js + React + TypeScript	Fast full-stack MVP, server-side data mutations, strong deployment path.	Vite React + Express if team already has template.
UI system	Tailwind + shadcn/ui + Framer Motion	Fast polished dashboard and animated launch-control feel.	Plain Tailwind components.
Database	Supabase Postgres	Relational model fits reservations, conflicts, audit logs, guest registrations, and RLS.	Local Postgres with Prisma/ Drizzle.
Realtime	Supabase Realtime	Live dashboard and check-in updates can subscribe to Postgres changes.	Manual refresh if realtime setup fails.
AI	OpenAI Structured Outputs	Reliable JSON extraction and schema validation for event intake.	Seeded parsed request fallback.
Decision Graph	React Flow	Built for node-based UIs and visualizations.	Custom SVG nodes for the main demo.
Twin	React SVG	Fast, controllable, data-driven, reliable in browser.	Static SVG image with clickable overlays.

Documents	HTML/PDF preview	Proposal preview is enough for demo; PDF optional.	HTML-only proposal page.
QR	qrcode package	Simple QR pass and asset QR generation.	Static QR image with route token.

External reference notes

The design uses current official documentation patterns where possible. OpenAI Structured Outputs supports schema-constrained responses, including function-calling and response-format flows. Supabase Realtime supports listening to Postgres database changes for live dashboards. Next.js Server Actions allow server-side mutations to be invoked from forms and client components. React Flow is a suitable library for node-based UIs such as the Decision Graph. These references are listed in the appendix.

12. AI architecture, guardrails, and prompt contracts

AI is a copilot, not the source of truth. It should read and explain structured data but never independently decide availability, price, or inventory. All critical operational facts must be calculated or loaded from the database.

AI responsibilities vs deterministic responsibilities

Area	AI may do	Deterministic code must do
Event intake	Extract structured fields from messy text.	Validate schema, normalize units, request missing fields.
Proposal	Write human-friendly proposal language.	Calculate prices, taxes, discounts, asset availability.
Conflict explanation	Summarize conflict and suggested fix in plain language.	Detect conflict and generate allowed fixes.
Task generation	Draft readable task descriptions.	Set deadlines, owners, dependencies, and statuses.
Decision Graph	Summarize graph meaning.	Create graph nodes and edges from planning outputs.
Guest content	Draft public description and agenda wording.	Expose only published fields and approved agenda data.

Structured intake schema

```
type EventIntake = {
  eventTitle?: string;
  eventType: 'conference' | 'workshop' | 'hackathon' | 'exhibition' | 'performance' | 'private' |
'other';
  expectedGuests: number;
  datePreference?: string;
  startTime?: string;
  endTime?: string;
  setupHours?: number;
  teardownHours?: number;
  needs: {
    mainStage: boolean;
    breakoutRooms: number;
    coffeeArea: boolean;
    registrationDesk: boolean;
    publicGuestRegistration: boolean;
    screens: number;
    projectors: number;
    wirelessMicrophones: number;
    wiredMicrophones: number;
    chairs: number;
    tables: number;
    speakers: number;
    livestream: boolean;
  };
  budget?: number;
  notes: string[];
  missingFields: string[];
  confidence: number;
};
```

AI orchestration flow

```
Organizer request text (submitted via portal or simulated inbox)
  -> AI structured extraction
  -> Zod validation
  -> normalization rules
  -> staff review
  -> deterministic planning engine
  -> plan snapshot
```

```
  -> AI explanation/proposal/task copy using plan snapshot only
  -> post-generation validation against allowed facts
```

Prompt guardrails

- The system prompt must state that the model cannot invent availability, prices, asset counts, spaces, or approvals.
- The model should be given a compact JSON snapshot of facts and asked to write explanation only.
- All AI-generated JSON must be validated with Zod or equivalent before use.
- All AI runs should be logged with input hash, output JSON, model, latency, and validation result.
- For demo safety, provide a deterministic fallback output for the primary request.

13. Domain model and lifecycle states

Core domain objects

Object	Definition	Owned by	Key relationships
EventOrganizer	External client (not Pyramid staff) who submits and tracks an event request. Authenticates as an external account linked to a client/contact.	Self (external)	Client, contact, event requests, proposals shared with them.
EventRequest	Initial inquiry before confirmation.	Submitted by Event Organizer; reviewed by Event Manager.	Organizer, client, extracted requirements, AI runs, possible events.
Event	Operational event being planned/executed.	Event Manager/Ops	Spaces, assets, tasks, quote, publication, registrations.
Space	Physical or informal zone.	Admin	Reservations, adjacency, Twin zone, tasks.
Asset	Individually tracked equipment item.	Inventory	Reservations, movements, issues, QR.
AssetBatch	Bulk quantity of same category.	Inventory	Batch reservations, movements.
AssetKit	Reusable bundle of category requirements.	Inventory/Ops	Kit items, event requirements.
Conflict	Detected issue affecting feasibility or launch readiness.	Ops/Engine	Suggestions, fixes, audit logs.
Task	Operational work item.	Ops/Team	Event, space, asset, owner, dependencies.
Quote	Client-facing pricing object.	Finance/Event Manager	Quote items, proposal, approvals.
EventPublication	Public event page derived from approved event.	Event Manager	Agenda, registrations, tickets, guest map.
GuestRegistration	Guest signup for a public event.	Guest	Ticket, check-in, event publication.
PlanVersion	Snapshot of an event plan.	System	Diffs, audit trail, rollback potential.

Event lifecycle

```
request_received -> parsed -> reviewed -> planning -> proposed -> confirmed -> published -> launch_ready
-> live -> completed -> archived
                \-> rejected
                \-> cancelled
```

Reservation lifecycle

```
none -> soft_hold -> reserved -> picked -> in_transit -> in_use -> returned -> released
                \-> cancelled
                \-> missing -> needs_inspection -> available
```

Guest registration lifecycle

```
registered -> ticket_issued -> checked_in -> attended
registered -> cancelled
registered -> no_show
```

14. Database design overview

The database is designed around Postgres and Supabase. The schema uses UUID primary keys, org_id for future multi-tenant isolation, timestamp columns, explicit enums, and enough auditability to explain decisions. The implementation can use SQL migrations directly, Drizzle, Prisma, or Supabase SQL editor. SQL is included separately in pyramid_os_schema.sql.

Entity relationship overview

```
organizations
  -> profiles -> profile_roles
  -> clients -> contacts        (an Event Organizer is an external account linked here)
  -> spaces -> space_adjacencies -> locations
  -> event_requests -> ai_extractions -> events
  -> events -> event_plan_versions -> plan_diffs
  -> events -> space_reservations
  -> events -> asset_reservations -> assets / asset_batches
  -> events -> conflicts -> conflict_suggestions
  -> events -> tasks -> task_assignments / task_dependencies
  -> events -> quotes -> quote_items -> proposals
  -> events -> event_publications -> agenda_items -> guest_registrations -> tickets -> checkins
  -> audit_logs / notifications / documents
```

Design decisions

- Use org_id everywhere to support future multi-organization usage even if hackathon uses one organization.
- Model the Event Organizer as an external account linked to a client/contact, so the outsider who submits the event is separate from internal staff and is scoped to their own data by RLS.
- Use time ranges for reservations to prevent setup/teardown conflicts, not only event overlaps.
- Track individual high-value assets and bulk batches separately to match real operations.
- Keep event_plan_versions as JSON snapshots because the plan is a composed view across many tables.
- Store AI runs for traceability and debugging, but do not make business logic depend on unvalidated AI output.
- Use event_publications as a public projection of Event data so Guest Mode can stay safe and controlled.
- Use audit_logs for human and system actions that matter for accountability.

15. Detailed data model: organizations, users, and permissions

Table	Purpose	Important fields	Notes
organizations	Top-level tenant/workspace.	id, name, slug, timezone, settings_json.	Single org for MVP; enables production separation.
profiles	Application profile linked to auth user (staff and external organizers).	id, org_id, full_name, email, avatar_url, status, is_external.	id can reference Supabase auth.users. is_external = true marks Event Organizer accounts.
roles	Role catalog.	id, code, name, description.	Seed admin, event_manager, ops_manager, inventory_manager, technician, finance, **event_organizer**, guest.
profile_roles	Many-to-many user roles.	profile_id, role_id, org_id.	Allows multi-role staff. event_organizer and guest are external and not mixed with staff roles.
teams	Operational groups.	ops, av, inventory, cleaning, security.	Used for task assignment and load balancing later.
team_members	Users inside teams.	team_id, profile_id, role_on_team.	Optional for hackathon; useful for staff tasks.

Event Organizer accounts: the organizer is an external profile (is_external = true) carrying the `event_organizer` role and linked to a `clients`/`contacts` row. Row Level Security scopes them to their own client_id so an organizer can only see and act on their own requests and the proposals shared with them.

16. Detailed data model: spaces and locations

Table	Purpose	Important fields	Notes
spaces	Bookable or operational zones.	name, floor, kind, capacity_min, capacity_max, allowed_layouts, public_visible.	Blue, Orange, Green, Yellow, Entrance, Corridors, Storage, Tech Storage.
space_adjacencies	Graph edges between spaces.	from_space_id, to_space_id, distance_weight, noise_transfer, guest_flow_weight.	Used by matcher, acoustic conflict, routes, asset flows.
locations	Asset storage and scan locations.	space_id, code, name, kind.	Can map to spaces but support shelves/corners.
space_reservations	Time-bound reservation of space.	event_id, space_id, reserved_from, reserved_until, status, purpose.	Covers setup, event, teardown, buffer.
space_layout_templates	Optional layout presets.	space_id, event_type, layout_name, capacity, setup_minutes.	Stretch: conference/workshop/ performance templates.

17. Detailed data model: events and planning

Table	Purpose	Important fields	Notes
event_requests	Incoming inquiry.	client_id, organizer_profile_id, raw_text, status, requested_date_text, extracted_json.	Submitted by an Event Organizer; AI intake starts here.
event_request_messages	Messages from email/ form/WhatsApp simulation.	request_id, channel, sender, body.	Supports fragmented communication replacement story.
events	Confirmed/planned event object.	title, event_type, expected_guests, start_at, end_at, setup_start_at, teardown_end_at, status.	Created after request review.
event_requirements	Normalized requirement rows.	event_id, requirement_type, key, quantity, unit, source.	Example: wireless_mics quantity 4.
event_plan_versions	JSON snapshots of full plan.	event_id, version_number, snapshot_json, created_by, label.	Used for Change Impact and rollback potential.
event_plan_diffs	Diffs between versions.	event_id, from_version_id, to_version_id, diff_json.	Can be generated on change.

18. Detailed data model: assets and equipment

Table	Purpose	Important fields	Notes
asset_categories	Equipment category catalog.	name, is_bulk, default_location_id, replacement_category_id, requires_inspection.	Chairs, mics, screens, cables.
assets	Serialized equipment.	category_id, name, serial_number, qr_code, current_location_id, status, condition.	Wireless Mic 03, Projector 01.
asset_batches	Bulk asset stacks.	category_id, name, quantity_total, quantity_available, qr_code, current_location_id.	Chair Stack A: 80.
asset_kits	Reusable setup bundles.	name, event_type, description.	Conference Kit, Cable Safety Kit.
asset_kit_items	Kit item requirements.	kit_id, category_id, quantity, optional, notes.	Conference Kit includes 150 chairs, 3 mics.
asset_reservations	Time-bound asset reservations.	event_id, asset_id, asset_batch_id, category_id, quantity, reserved_from, reserved_until, status.	Supports individual and batch reservations.
asset_movements	Asset chain of custody.	asset_id/batch_id, from_location_id, to_location_id, moved_by, moved_at.	QR scan updates this.
asset_issues	Damage/missing/ maintenance notes.	asset_id, issue_type, severity, description, status.	Feeds launch readiness.

19. Detailed data model: conflicts, tasks, quotes, guests

Table	Purpose	Important fields	Notes
conflicts	Detected operational issues.	event_id, conflict_type, severity, status, detected_by, description.	Capacity, mic shortage, cable risk, overlap.
conflict_suggestions	Allowed fixes.	conflict_id, action_type, title, payload_json, risk_after.	Apply fix can update reservations/tasks.
tasks	Operational work items.	event_id, space_id, asset_id, title, role, due_at, status.	Setup, sound check, registration, teardown.
task_dependencies	Task ordering.	task_id, depends_on_task_id.	Sound check depends on AV setup.
quotes	Price container.	event_id, status, subtotal, discount_total, tax_total, total.	Deterministic calculations.
quote_items	Price lines.	quote_id, item_type, description, quantity, unit_price, total.	Room, equipment, staff, cleaning.
event_publications	Public event page projection.	event_id, slug, title_public, description_public, status, capacity_public.	Guest-safe projection.
agenda_items	Public agenda schedule.	publication_id, space_id, title, starts_at, ends_at.	Used in guest map.
guest_registrations	Guest signup.	publication_id, guest_id, status, ticket_id.	Registration count.
guest_tickets	QR pass.	registration_id, ticket_token, qr_payload, status.	Check-in token.
guest_checkins	Arrival record.	ticket_id, checked_in_at, checked_in_by.	Updates dashboard.

20. Core SQL DDL excerpt

The full SQL migration is exported as pyramid_os_schema.sql. The excerpt below shows the style of the schema and the main reservation/asset/event patterns. Engineers should use the separate SQL file as the source of truth for implementation.

```
-- Pyramid OS hackathon MVP database schema
-- Target: Supabase Postgres
-- Notes: This schema is intentionally explicit for engineering alignment.

create extension if not exists pgcrypto;
create extension if not exists btree_gist;

-- ---------- Enums ----------
do $$ begin
  create type app_user_status as enum ('active', 'invited', 'disabled');
exception when duplicate_object then null; end $$;

do $$ begin
  create type event_request_status as enum ('received', 'parsed', 'reviewed', 'planning',
    'proposed', 'approved', 'rejected', 'cancelled');
exception when duplicate_object then null; end $$;

do $$ begin
  create type event_status as enum ('draft', 'planning', 'proposed', 'confirmed',
    'published', 'launch_ready', 'live', 'completed', 'archived', 'cancelled');
exception when duplicate_object then null; end $$;

do $$ begin
  create type reservation_status as enum ('soft_hold', 'reserved', 'picked', 'in_transit',
    'in_use', 'returned', 'released', 'cancelled');
exception when duplicate_object then null; end $$;

do $$ begin
  create type asset_status as enum ('available', 'soft_hold', 'reserved', 'picked',
    'in_transit', 'in_use', 'returned', 'needs_inspection', 'maintenance', 'missing', 'retired');
exception when duplicate_object then null; end $$;

-- (additional enums: asset_condition, conflict_severity, conflict_status,
--  task_status, approval_status, publication_status, ticket_status)

-- ---------- Organization and users ----------
create table if not exists organizations (
  id uuid primary key default gen_random_uuid(),
  name text not null,
  slug text not null unique,
  timezone text not null default 'Europe/Tirane',
  settings_json jsonb not null default '{}'::jsonb,
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now()
);

create table if not exists profiles (
  id uuid primary key,
  org_id uuid not null references organizations(id) on delete cascade,
  full_name text not null,
  email text not null,
  avatar_url text,
  is_external boolean not null default false,   -- true for Event Organizer accounts
  status app_user_status not null default 'active',
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now(),
  unique(org_id, email)
);

create table if not exists roles (
  id uuid primary key default gen_random_uuid(),
  code text not null unique,                    -- includes 'event_organizer'
```

```
  name text not null,
  description text
);

create table if not exists profile_roles (
  profile_id uuid not null references profiles(id) on delete cascade,
  role_id uuid not null references roles(id) on delete cascade,
  org_id uuid not null references organizations(id) on delete cascade,
  created_at timestamptz not null default now(),
  primary key(profile_id, role_id)
);

-- ---------- Clients (the external organizer's company) ----------
create table if not exists clients (
  id uuid primary key default gen_random_uuid(),
  org_id uuid not null references organizations(id) on delete cascade,
  name text not null,
  client_type text not null default 'organization',
  notes text,
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now()
);

create table if not exists contacts (
  id uuid primary key default gen_random_uuid(),
  org_id uuid not null references organizations(id) on delete cascade,
  client_id uuid references clients(id) on delete cascade,
  organizer_profile_id uuid references profiles(id),  -- links the external organizer account
  full_name text not null,
  email text,
  phone text
);
```

The full SQL file includes all tables, enums, indexes, and triggers described in this documentation. It is intentionally designed to be read and edited by the backend engineer during implementation.

21. API and service design

The MVP can use Next.js Server Actions for most authenticated staff mutations and Route Handlers for public/guest and organizer endpoints, QR tickets, and AI orchestration. The API names below are implementation contracts that engineers can map to Server Actions or REST endpoints.

Organizer API contracts

Operation	Route / action	Input	Output
Submit event request	POST /api/organizer/requests	rawText, sourceChannel, organizer identity	requestId, status
Track request status	GET /api/organizer/requests/[id]	requestId (own)	request status, shared proposal status
Review/approve proposal	POST /api/organizer/requests/[id]/approve	requestId, decision, comments	approval record, updated status

Staff API contracts

Operation	Route / action	Input	Output
Parse event request	POST /api/ai/parse-event-request	rawText, sourceChannel	EventIntake JSON + confidence + missing fields
Create event from request	createEventFromRequest()	requestId, reviewedFields	eventId, planVersionId
Generate plan	generateEventPlan()	eventId	space matches, asset requirements, conflicts, quote draft
Apply conflict fix	applyConflictFix()	conflictId, suggestionId	updated conflicts, reservations, tasks, launch gates
Reserve assets	reserveAssets()	eventId, requirements, reservationType	asset reservations, shortages
Generate proposal	generateProposal()	eventId, quoteId	proposal document preview/storage path
Publish event	publishEvent()	eventId, public fields, agenda	eventPublication, public URL
Scan asset	scanAssetQr()	qrCode, eventId, action	asset status, movement record
Check launch readiness	getLaunchReadiness()	eventId	gate statuses and blockers

Guest API contracts

Operation	Route	Input	Output
List public events	GET /api/public/events	filters optional	published event cards
Event detail	GET /api/public/events/[slug]	slug	public event details, agenda, guest route
Register guest	POST /api/public/events/[slug]/register	guest form fields	registration and ticket token
Get ticket	GET /api/public/tickets/[token]	ticket token	guest ticket, event summary, QR payload
Check in ticket	POST /api/staff/check-in	ticket token, location	check-in record and count update

Example: parse request response

```
{
  "eventType": "conference",
  "expectedGuests": 180,
```

```
  "datePreference": "next month",
  "setupHours": 2,
  "needs": {
    "mainStage": true,
    "breakoutRooms": 2,
    "coffeeArea": true,
    "registrationDesk": true,
    "publicGuestRegistration": true,
    "screens": 1,
    "projectors": 1,
    "wirelessMicrophones": 4,
    "wiredMicrophones": 0,
    "chairs": 150,
    "tables": 15,
    "speakers": 2,
    "livestream": false
  },
  "missingFields": ["exact event date", "event end time"],
  "confidence": 0.92
}
```

22. Core algorithms and planning logic

Space matching algorithm

```
score(space, event):
  capacity_score    = fit(expected_guests, space.capacity_min, space.capacity_max)
  availability_score= 1 if no overlapping reservation else 0
  layout_score      = 1 if requested layout in allowed_layouts else 0.5
  adjacency_score   = bonus if space is near registration, breakouts, or storage needs
  setup_score       = 1 if setup window >= space.setup_time_minutes else 0.3
  guest_flow_score  = reduce if corridor or entrance pressure is high
  total = 0.30*capacity_score + 0.25*availability_score + 0.15*layout_score
        + 0.10*adjacency_score + 0.10*setup_score + 0.10*guest_flow_score
```

Feasibility score

Item	Decision / Requirement
Space fit — 30%	Capacity, multi-space plan, availability, setup compatibility.
Asset readiness — 25%	Required assets available, substitutes available, QR/scanning status.
Schedule safety — 15%	Setup and teardown buffers, overlapping events, return windows.
Power/cable readiness — 10%	Cable kit, power points, guest path cable risk.
Staff/task readiness — 10%	Tasks generated, owners assigned, blockers absent.
Guest readiness — 10%	Guest page, registration, map, capacity, check-in plan.

Conflict detection rules

Conflict type	Trigger	Severity suggestion	Auto-fix options
Space overlap	Requested time range overlaps reserved space.	critical if exact required space; high if alternative exists.	Alternative space, time shift, split agenda.
Asset shortage	Required quantity exceeds available quantity.	medium to critical depending asset category.	Replacement category, external rental, reduce requirement.
Serialized double-booking	Same asset has overlapping reservation.	high for AV; medium for normal equipment.	Alternate asset, substitute, shift time.
Setup conflict	Setup starts before previous teardown/ return buffer ends.	high.	Increase buffer, add crew, shift event.
Power/cable risk	Powered devices exceed zone capacity or cable path crosses guest path.	medium/high.	Add cable kit, cable covers, route along wall.
Guest flow risk	Arrival or transition count exceeds comfort capacity.	medium/high.	Open check-in earlier, split coffee, add signage.

Auto-fix algorithm

```
for conflict in open_conflicts:
  if conflict.type == 'asset_shortage':
    find replacement_category_id
    if replacement available:
      create suggestion(substitute_asset)
    else:
      create suggestion(external_rental_request)
  if conflict.type == 'power_cable':
    if Cable Safety Kit available:
      create suggestion(reserve_cable_kit)
```

```
  if conflict.type == 'space_capacity':
    find adjacent overflow space or larger alternative
rank suggestions by residual risk, cost, and disruption
```

Event DNA scoring

DNA dimension	Inputs	Example high condition
People intensity	expected_guests, registration capacity, arrival window.	Guests > 150 or arrival window < 30 minutes.
Technical complexity	mics, screens, speakers, livestream, power devices.	AV categories >= 4.
Space complexity	number of spaces, adjacency, spillover.	Main + 2 breakouts + coffee.
Asset intensity	asset quantity and category count.	More than 5 categories or >150 items.
Guest journey complexity	agenda movement, route count, check-in pressure.	Guests move between 3+ zones.
Setup risk	setup window vs calculated setup minutes.	Available setup < required setup.
Brand value	public event, strategic audience, sponsor potential.	Tech/startup/public event.
Operational risk	conflicts, unresolved gates, low buffers.	Any critical conflict.

Launch readiness gate logic

```
Gate states:
  GO      = no blockers and all required records complete
  WARNING = non-critical conflict exists or missing optional confirmation
  BLOCKED = critical conflict, missing approval, unavailable required asset, or unpublished guest page

EVENT READY FOR LAUNCH if:
  all critical gates are GO
  no open critical conflicts
  proposal approved or internal override exists
  space and asset reservations are confirmed
  guest page/map/check-in plan is ready when event is public
```

23. Frontend implementation details

The UI should feel like a command center. Avoid generic admin navigation labels. Use stages: Understand, Simulate, Protect, Explain, Launch, Guest. This lets the demo feel like the event is being assembled live. The external organizer uses a separate, minimal portal that does not expose these internal stages.

Recommended route structure

```
/app
  /(organizer)/request                -- external organizer: submit request
  /(organizer)/request/[id]           -- track status, review/approve proposal
  /(staff)/dashboard
  /(staff)/events/[eventId]/understand
  /(staff)/events/[eventId]/simulate
  /(staff)/events/[eventId]/protect
  /(staff)/events/[eventId]/explain
  /(staff)/events/[eventId]/launch
  /(staff)/inventory
  /(staff)/inventory/[assetQr]
  /(public)/events
  /(public)/events/[slug]
  /(public)/tickets/[token]
  /(public)/guest-map/[slug]
```

Critical components

Component	Purpose	Inputs	Notes
OrganizerRequestPortal	External organizer submits request and tracks status.	rawText, requestStatus, sharedProposal.	No internal data; external auth only.
EventIntakePanel	Messy request input and extraction review.	rawText, extractedJson.	Must have loading and fallback.
EventDNACard	Visual operational fingerprint.	event_dna_json.	Radar/bars with explanations.
PyramidTwin	Main map with modes and layers.	zones, reservations, conflicts, flows, routeSteps.	SVG and Framer Motion.
SpaceMatchCards	Show ranked space recommendations.	spaceScores.	Must show reasons.
InventoryPlanner	Asset requirements and reservations.	requirements, availability, reservations.	Show missing and replacement.
ConflictCenter	List conflicts and auto-fixes.	conflicts, suggestions.	Apply fix action.
DecisionGraph	Explain plan.	nodes, edges.	React Flow or custom SVG.
ChangeImpactDiff	Before/after diff.	planVersionA, planVersionB.	Main demo: guests 180 to 240.
LaunchMode	GO/WARNING/BLOCKED gates.	readinessGates.	Final emotional screen.
GuestEventCard	Public event browsing.	publication summary.	Mobile-friendly.
GuestTicket	QR pass and agenda.	ticket token, registration.	Use qrcode.

Frontend state rules

- Do not rely on client-side-only calculations for authoritative reservations or pricing.
- Use optimistic UI only for non-critical transitions such as opening panels or marking read notifications.
- After applying conflict fixes, refetch or invalidate event plan state so all screens agree.

- Guest Mode and the organizer portal must be available without staff auth but only for published events, own requests, and token-protected tickets.
- All launch gates should derive from one server-side readiness function to avoid inconsistent statuses.

24. Realtime dashboards and notifications

Realtime is useful for the Pyramid Twin, inventory movements, tasks, conflicts, and guest check-ins. For the hackathon, realtime can be limited to check-in count and asset/task state updates.

Realtime channels

Channel	Events listened to	Affected UI
event:{eventId}:plan	space_reservations, conflicts, tasks, asset_reservations.	Pyramid Twin, Launch Mode, Protect page.
event:{eventId}:checkins	guest_checkins, guest_registrations.	Check-in dashboard, Guest readiness gate.
inventory	asset_movements, asset_issues, asset status updates.	Inventory dashboard and asset layer.
tasks:{eventId}	tasks inserts/updates.	Operations board.

Notification examples

- New organizer request received: Startup Conference for 180 guests submitted by an external organizer.
- Asset conflict detected: one wireless microphone unavailable for Startup Conference.
- Fix applied: Wired Mic 01 substituted and reserved.
- Proposal shared: proposal sent to the Event Organizer for approval.
- Guest page published: registration open for AI Startup Conference.
- Check-in pressure warning: 70 guests expected in next 15 minutes.
- Launch gate changed: Power moved from WARNING to GO.

25. Security, RBAC, privacy, and auditability

Security model

- Use Supabase Auth or an equivalent auth layer for staff and external organizer accounts.
- Use profile_roles for role checks. Enforce checks on the server; do not rely on hidden UI only.
- Event Organizers are external accounts scoped to their own client_id; they can submit and track only their own requests and view proposals shared with them.
- Use public routes only for published event_publications and tokenized tickets.
- Use signed/random high-entropy ticket_token values for QR tickets. QR codes should not contain private guest data.
- Use audit logs for plan generation, reservation, conflict fix, launch gate override, publication, and check-in actions.
- Do not expose exact asset storage locations to guests or organizers.

Example RLS policy direction

```
-- Staff can read records in their organization if they have an active profile.
-- Event Organizers can read/create only their own event_requests (matched by client_id/
organizer_profile_id),
--   and read proposals/quotes explicitly shared with them. No inventory, conflicts, or pricing rules.
-- Guests can read only event_publications where status = published.
-- Guests can access their ticket only by unguessable ticket_token route handler.
-- Asset and conflict tables are staff-only.
-- Audit logs are insert-only by app service functions; direct updates should be restricted.
```

Privacy risk register

Risk	Impact	Mitigation
Organizer sees another client's request or pricing rules	Privacy/commercial breach.	Scope organizer access to own client_id via RLS; never expose internal pricing rules.
Guest emails exposed publicly	Privacy breach.	Guest list staff-only; ticket route tokenized.
Storage rooms exposed on guest map	Security/operational risk.	Guest Mode uses public_visible=false filter.
AI leaks internal notes into public description	Reputation/privacy issue.	AI public content uses event_publication allowed fields only.
Unauthorized conflict fix	Operational disruption.	Server-side role check and audit log.
QR ticket guessing	Fraud/check-in abuse.	Use UUID/random token, not sequential IDs.

26. Testing, QA, and demo reliability

Core scenario tests

Test ID	Scenario	Expected result
T-000	External organizer submits request via portal.	Request created, visible to staff, status trackable by organizer only.
T-001	Submit startup conference messy request.	Structured intake returns expectedGuests 180 and requirements.
T-002	Generate plan from reviewed request.	Green, Blue, Yellow, Entrance recommended.
T-003	Wireless Mic 04 already reserved.	Conflict created for one missing wireless mic.
T-004	Apply wired mic substitution.	Conflict resolved, asset reservation created, Launch Assets gate GO.
T-005	Add Cable Kit A.	Power gate moves from WARNING to GO.
T-006	Change guests 180 to 240.	Diff shows guests, chairs, quote, feasibility, conflict changes.
T-007	Publish event.	Public event page appears and guest registration opens.
T-008	Register guest.	Registration, QR ticket, and remaining capacity update.
T-009	Check in ticket.	Check-in dashboard increments.
T-010	Open Guest Map.	Only public spaces and route steps are visible.

Fallback plan for live demo

- If AI API fails, use pre-seeded extracted JSON and say the extraction is cached for demo reliability.
- If PDF export fails, show proposal preview page instead of downloadable PDF.
- If realtime fails, add a Refresh button and demonstrate check-in count update after refresh.
- If QR scanning fails due to camera permissions, click a simulated Scan Asset button with QR code prefilled.
- If the Decision Graph library causes problems, use a static SVG graph for the main scenario.
- If time is tight, simplify full Git for Events to one before/after Change Impact diff.

Seed data requirements

Item	Decision / Requirement
Spaces	Green 200, Orange 120, Blue 80, Yellow 80, Entrance 120 standing, Corridor 80 flow comfort, Storage A, Storage B, Tech Storage.
Organizer	One external organizer account (e.g. "AI Startup Org") linked to a client, used to submit the main demo request.
Events	Startup Conference main demo; Robotics Workshop as conflict event; Digital Art Exhibition as public event card.
Assets	220 chairs, 30 tables, 4 wireless mics, 2 wired mics, 2 screens, 2 projectors, 2 speakers, 6 extension cables, 4 cable covers.
Reservations	Wireless Mic 04 reserved for Robotics Workshop; Orange Room booked for another event.
Guests	Optional prefilled registrations to show capacity and check-in dashboard.

27. Deployment and environment setup

Recommended environments

Environment	Purpose	Data
Local	Fast development and component testing.	Seeded local or Supabase dev project.
Preview	Pull request / Vercel preview deployments.	Shared demo project with resettable seed script.
Demo	Stable URL for hackathon judging.	Locked seed data and fallback outputs.

Environment variables

```
NEXT_PUBLIC_SUPABASE_URL=
NEXT_PUBLIC_SUPABASE_ANON_KEY=
SUPABASE_SERVICE_ROLE_KEY=
OPENAI_API_KEY=
APP_BASE_URL=https://pyramid-os-demo.vercel.app
QR_SIGNING_SECRET=
DEMO_MODE=true
```

Build and deployment checklist

- Run database migration and seed script before final demo.
- Lock demo data by preserving conflict event and inventory counts.
- Verify public event pages do not require staff login.
- Verify the organizer portal only exposes the organizer's own request and shared proposal.
- Verify staff routes redirect unauthenticated users.
- Prepare fallback screenshots for every major screen.
- Record a short demo video in case network or projector fails.

28. Five-engineer implementation plan

The team should avoid building five separate mini-products. Each engineer owns a vertical but integrates into the same demo event. Daily integration is not enough in a 48-hour hackathon; integrate every few hours.

Engineer	Ownership	Deliverables
Engineer 1 — Frontend Command Center	Pyramid Twin, Event DNA, Launch Mode, layout, animations.	Understand/Simulate/Launch screens, Twin modes, polished UI shell.
Engineer 2 — Backend/ Data	Supabase schema, migrations, seed data, auth/RBAC (incl. external organizer role), query helpers.	Database running, seed scripts, server actions/data access.
Engineer 3 — AI/Docs	AI intake, proposal/ops pack text, explanations, staff briefings.	Structured extraction, generated proposal preview, explanation outputs.
Engineer 4 — Planning Engine	Space matching, equipment reservations, conflicts, auto-fixes, feasibility, launch gates.	Core deterministic functions and tests.
Engineer 5 — Guest/ Organizer/Demo/Polish	Organizer portal, guest browser, registration, QR, check-in, deck, video, final script.	Organizer + guest journey, demo story, fallback screenshots, final polish.

48-hour execution roadmap

Time	Goal	Output
0-4h	Agree on demo scenario, schema, route structure, UI theme.	Clickable design skeleton and seed data plan.
4-12h	Build data foundation and first screens.	Organizer request + event intake, spaces/assets seed, Twin skeleton, plan generation stub.
12-20h	Implement planning engine.	Space matching, inventory availability, conflict detection, quote draft.
20-28h	Add wow features.	Event DNA, auto-fix, Launch Mode, Decision Graph first version.
28-36h	Add Guest Mode and QR.	Public event page, registration, ticket, check-in counter, guest map.
36-42h	Integrate and polish.	End-to-end flow, fallback states, animations, bug fixes.
42-48h	Demo hardening.	Deck/video, rehearsals, reset script, backup screenshots.

Definition of done for MVP

- A single demo event can complete the full flow without manual database editing.
- An external organizer can submit the request and a staff user can take it through to launch.
- Every screen in the demo has seeded data and a loading/error fallback.
- At least one real server-side function is used for planning, conflict detection, and auto-fix.
- The guest registration creates actual records and affects the check-in dashboard.
- The final Launch Mode screen is visually polished and reachable in under three minutes from demo start.
- The pitch line and product framing are consistent across app, deck, and presenter script.

29. Post-hackathon roadmap

Phase	Focus	Features
Phase 1 — Production pilot	Reliability and real venue data.	Import real spaces/assets, real staff roles, organizer self-service accounts, proposal templates, basic notifications.
Phase 2 — Operational intelligence	Learning from every event.	Pyramid Memory, event replay, no-show prediction, crowd flow heatmap, supplier requests.
Phase 3 — Advanced twin	More visual and real-time operations.	3D Spline/Three.js twin, live staff mobile app, kiosk mode, more route logic.
Phase 4 — Physical tracking	Hardware-assisted asset tracking.	NFC/RFID, barcode printers, maintenance checklists, asset depreciation.
Phase 5 — Ecosystem	Revenue and client experience.	Payments, external supplier marketplace, sponsor/booth allocator, analytics dashboard.

30. Appendix A: Prompt templates

System prompt for event intake

```
You extract venue event requirements for Pyramid OS. Return only JSON matching the provided schema.
Do not invent availability, price, inventory, or approval status. If a value is unclear, add it to
missingFields.
Normalize quantities when possible. Infer common event needs only when strongly implied by the text.
```

Proposal generation prompt pattern

```
You write a client-facing proposal using only the validated plan facts below.
Do not change prices, spaces, quantities, availability, or dates.
Use professional, concise language. Include assumptions and next steps.
FACTS_JSON: {{planSnapshot}}
QUOTE_JSON: {{quote}}
```

Conflict explanation prompt pattern

```
Explain this operational conflict and the allowed suggested fix in plain language for non-technical venue
staff.
Do not propose any fix not listed in ALLOWED_FIXES.
CONFLICT_JSON: {{conflict}}
ALLOWED_FIXES: {{suggestions}}
```

31. Appendix B: Seed data specification

Seed group	Records
Organizers	One external Event Organizer account ("AI Startup Org") linked to a client and used to submit the Startup Conference request.
Spaces	Green Room, Blue Room, Yellow Room, Orange Room, Entrance, Main Corridor, Lower Corridor, Storage A, Storage B, Tech Storage, Tech Booth.
Adjacency	Entrance-Main Corridor, Corridor-Blue, Corridor-Orange, Lower Corridor-Green, Lower Corridor-Yellow, Tech Storage-Green.
Assets	Wireless Mic 01-04, Wired Mic 01-02, Projector 01-02, Screen 01-02, Speaker 01-02, Chair Stack A-C, Table Stack A-B, Cable Kit A, Signage Kit.
Event conflict	Robotics Workshop reserves Wireless Mic 04 during Startup Conference setup/event window.
Pricing	Space rental, AV kit, technical support, cleaning/security, optional guest page.
Guest cards	AI Startup Conference, Robotics Workshop, Digital Art Exhibition, Youth Coding Bootcamp.

32. Appendix C: Decision log

Decision	Why	Alternative rejected
Model the Event Organizer as an explicit external role.	The person who sets the event is an outsider/client, not Pyramid staff; separating them clarifies permissions and matches the real workflow.	Treating the organizer as just a staff-entered field hides the real external actor.
Build 2D/SVG Twin instead of full 3D twin.	More reliable, data-driven, achievable in 48 hours.	Full Three.js/3D model too risky.
Use QR for assets instead of RFID/NFC.	QR is cheap, demoable, and realistic without hardware.	RFID/NFC requires tags/readers and more testing.
Add Guest layer.	Differentiates from internal-only booking tools and connects operations to visitor experience.	Internal-only product less memorable.
Sacrifice advanced Git first if needed.	Guest layer creates stronger full lifecycle story.	Full versioning is clever but less visible to jury.
AI writes/explains; code calculates.	Increases trust and avoids hallucinated operations.	AI-only planner would be risky and less credible.
Use plan snapshots for Change Impact.	Simpler than event-sourcing everything.	Full event sourcing too much for hackathon.

33. Appendix D: Reference sources for implementation choices

These sources inform technology choices and should be checked during implementation if exact API details are needed. The documentation does not require exact version-specific code from these sources; they are included for grounding and engineer reference.

Topic	Official source	Use in this plan
OpenAI Structured Outputs	https://developers.openai.com/api/docs/guides/structured-outputs	Schema-constrained event intake and structured AI outputs.
Supabase Realtime Postgres Changes	https://supabase.com/docs/guides/realtime/postgres-changes	Live updates for check-ins, reservations, tasks, and inventory.
Next.js Server Actions	https://nextjs.org/docs/app/api-reference/functions/server-actions	Server-side form/data mutations in the app.
React Flow	https://reactflow.dev	Decision Graph node-based visualization.

34. Appendix E: Glossary

Term	Definition
Event Organizer	External client (not Pyramid staff) who submits an event request and tracks it from inquiry to proposal approval. Distinct from the Event Manager (staff who reviews and runs the event) and from a Guest (an event attendee).
Pyramid OS	The proposed operating system for event planning, operations, and guest experience.
Launch Control	The main staff experience that turns an event plan into GO/NO-GO readiness.
Pyramid Twin	A simplified data-driven map of spaces, assets, conflicts, tasks, and guest routes.
Event DNA	A visual complexity fingerprint for an event.
Soft hold	Temporary reservation before final approval.
Confirmed reservation	Approved reservation that blocks assets/spaces for execution.
Guest Mode	Public-facing event discovery, registration, QR pass, and navigation layer.
Decision Graph	Visual explanation of why the plan was selected.
Auto-fix	Allowed system action that resolves a conflict by changing reservations/tasks/plan.
Launch gate	Readiness category such as Space, Assets, Power, Staff, Guest Page, or Map.

35. Appendix F: Detailed table-by-table engineering notes

organizations

Use one row for the Pyramid in the MVP. Keep timezone set to Europe/Tirane because all event scheduling, setup buffers, and guest registration windows depend on venue-local time. The settings_json field can hold defaults such as default setup buffer, default teardown buffer, and launch gate thresholds.

profiles and profile_roles

Do not overload auth.users with app-specific fields. Keep profile records in public.profiles and grant roles through profile_roles. This enables one user to be both Event Manager and Operations Manager during the hackathon without duplicating accounts. External Event Organizer accounts are marked with is_external = true and carry only the event_organizer role.

clients and contacts

The Event Organizer is the external party who submits the request. Model them as a client (their company) plus a contact, and link the contact to an external profile via organizer_profile_id so the organizer can authenticate, submit, and track their own requests.

spaces

Represent both formal rooms and informal operational areas. The challenge explicitly mentions spaces beyond the four main rooms; Entrance and Corridors should be first-class spaces because registration, coffee, and guest flow often depend on them.

space_adjacencies

Use this table to make the Pyramid more than a list of rooms. Adjacencies power breakout recommendations, guest routes, acoustic conflicts, asset movement difficulty, and crowd-flow warnings.

locations

A location is more granular than a space. Tech Storage can be a space, but a shelf or cabinet can be a location. For the MVP, one location per storage space is enough.

event_requests

Keep raw_text permanently because it is evidence of what the organizer asked. Store extracted_json and confidence but treat reviewed event fields as authoritative after staff confirmation. Record organizer_profile_id so the request is owned by its external submitter.

events

Use events as the operational truth. It is valid to create an event in draft or planning state before confirmation because planning generates quote, feasibility, and conflicts.

event_requirements

Rows are easier to query than only JSON. Keep both event_requirements for operations and event_dna_json for computed display metrics.

event_plan_versions

Use JSON snapshots for rapid implementation. A snapshot should include selected spaces, asset reservations, quote summary, conflicts, tasks, DNA, and launch gates. This allows easy diffing and demo reliability.

space_reservations

Always reserve full operational windows: setup_start_at to teardown_end_at, plus return buffer if needed. This prevents a second event from using a room while the first is still being cleared.

asset_categories

replacement_category_id is central to auto-fix. Wireless Microphone can point to Wired Microphone as a fallback. Cable Kit can point to External Rental category later.

assets

Use serialized rows only for valuable equipment. Microphones, screens, projectors, routers, cameras, and speakers should be individual assets.

asset_batches

Bulk assets like chairs and tables should be batched. Scanning every chair is unrealistic; scanning Chair Stack A is realistic.

asset_kits

Kits are the fast path for MVP and future scale. Conference Kit, Breakout Kit, Registration Kit, and Cable Safety Kit make AI recommendations look smart while using simple deterministic expansion.

asset_reservations

A reservation can point to an asset, a batch, or only a category during soft hold. In final reservation, prefer exact asset or batch IDs when possible.

asset_movements

This gives every asset a journey. It is also the proof that the system replaces institutional memory about where equipment is.

conflicts

Conflicts should be generated by deterministic rules and can be explained by AI. Keep severity and source_json so staff can understand what caused the issue.

conflict_suggestions

Every auto-fix should be represented as a suggestion with a payload. This makes it safe to preview the effect before applying it.

tasks

Tasks are generated from selected spaces, assets, event type, and conflicts. Do not hard-code one static task list; create templates per event type and inject requirements.

quotes and quote_items

Keep prices deterministic and auditable. AI can write proposal language, but quote_items are calculated from pricing_rules or hard-coded MVP rules.

event_publications

This table is the safety boundary between internal operations and public guest pages. Only fields copied here are public.

guest_tickets

QR payload should contain a random token or URL, not raw guest data. Staff check-in route resolves the token server-side.

ai_runs

Log AI runs for debugging. Include validation_status so engineers can tell whether a model response was accepted or rejected.

audit_logs

Audit logs should capture before/after JSON for critical actions such as applying fixes, approving quotes, publishing event, and changing reservations.

36. Appendix G: Implementation pseudocode library

```
export async function generatePlan(eventId: string) {
  const event = await loadEventWithRequirements(eventId);
  const spaces = await loadSpacesWithReservations(event.orgId, event.window);
  const inventory = await loadInventoryAvailability(event.orgId, event.window);

  const spaceMatches = scoreSpaces(event, spaces);
  const selectedSpaces = chooseMultiSpacePlan(event, spaceMatches);
  const assetRequirements = buildAssetRequirements(event, selectedSpaces);
  const assetPlan = reserveAssetsDryRun(event, assetRequirements, inventory);
  const conflicts = detectConflicts(event, selectedSpaces, assetPlan);
  const suggestions = buildConflictSuggestions(conflicts, inventory, selectedSpaces);
  const quote = buildQuote(event, selectedSpaces, assetPlan);
  const tasks = buildTasks(event, selectedSpaces, assetPlan, conflicts);
  const eventDna = computeEventDna(event, selectedSpaces, assetPlan, conflicts);
  const readiness = computeLaunchReadiness(event, selectedSpaces, assetPlan, conflicts, tasks);

  return { spaceMatches, selectedSpaces, assetRequirements, assetPlan, conflicts,
           suggestions, quote, tasks, eventDna, readiness };
}

export function applySubstitutionFix(conflict, suggestion, plan) {
  assert(conflict.status === 'open');
  assert(suggestion.actionType === 'substitute_asset');
  const { originalCategoryId, replacementCategoryId, quantity } = suggestion.payload;
  const replacement = findAvailableAssets(replacementCategoryId, quantity, plan.window);
  if (!replacement.sufficient) throw new Error('Replacement no longer available');
  plan.assetReservations.push(...replacement.reservations);
  plan.tasks.push(makeTask('Update AV setup with substituted equipment'));
  conflict.status = 'auto_fixed';
  conflict.resolvedAt = new Date().toISOString();
  plan.audit.push({ action: 'apply_conflict_fix', conflictId: conflict.id, suggestionId:
suggestion.id });
  return recomputeReadiness(plan);
}

export function computeGuestCapacity(publication) {
  const capacity = publication.capacityPublic;
  const registered = publication.registrations.filter(r => r.status !== 'cancelled').length;
  const checkedIn = publication.checkins.length;
  const remaining = Math.max(capacity - registered, 0);
  const checkInRate = registered > 0 ? checkedIn / registered : 0;
  return { capacity, registered, checkedIn, remaining, checkInRate };
}
```

37. Appendix H: Demo script and reset checklist

Presenter script

Opening: The Pyramid does not need another calendar. It needs launch control for physical experiences. Pyramid OS connects backstage intelligence with frontstage guest experience.

Demo step 1: Acting as an external event organizer, paste the startup conference request into the organizer portal and show AI Event Intake extracting the requirements.

Demo step 2: Switch to the staff view. Show Event DNA and say that before booking a space, the system understands the operational fingerprint of the event.

Demo step 3: Open Pyramid Twin. Green is the keynote, Blue and Yellow are breakouts, Entrance is registration and coffee.

Demo step 4: Open Conflict Center. The system detects that only three wireless microphones are available and auto-fixes with one wired microphone.

Demo step 5: Open Power/Cable Intelligence. The system catches missing cable covers and adds Cable Kit A.

Demo step 6: Open Decision Graph to explain why the plan was chosen.

Demo step 7: Change guests from 180 to 240 and show the Change Impact diff.

Demo step 8: Generate proposal and operations pack preview, and share the proposal back to the organizer.

Demo step 9: Open Launch Mode and end with EVENT READY FOR LAUNCH.

Demo step 10: Publish to Guest Mode, register as guest, show QR pass, guest map, and check-in counter update.

Reset checklist

- Database seed script has been run.
- External organizer account exists and can submit the demo request.
- Startup Conference exists in draft/planning state.
- Robotics Workshop reserves Wireless Mic 04.
- Guest registrations for Startup Conference are reset or set to known count.
- No critical conflicts are pre-resolved unless demo starts after conflict center.
- Presentation browser tabs are logged in and public guest page is accessible.
- Fallback screenshots and recorded video are available offline.